

Project 1

Chess Tournament Details

Approach

I would start by cleaning the input file by removing everything that does not constitute the information about any particular player in the tournament: the header of the file, as well as the separator lines. At this point, the file consists of two lines for each player where the first line contains the name, total points, and details of how every single round went for him/her and the second line – his/her state and rating.

I extract the data from every single line using “|” as a delimiter because of its predictability. The rating of some players is followed by additional comments that I strip using regular expressions to get only the number. I also extract the name of each player’s opponent in every round using the same technique and skipping rounds when he/she got a bye or did not participate.

Now I match the player number to his/her rating and calculate the average rating of opponents for each individual player by averaging the ratings of all the players whom he/she played against. Finally, I combine all the required information into one table.

Code Base

```
library(stringr)
library(dplyr)
```

```
Attaching package: 'dplyr'
```

```
The following objects are masked from 'package:stats':
```

```
filter, lag
```

```
The following objects are masked from 'package:base':
```

```
intersect, setdiff, setequal, union
```

```
data_url <- "https://raw.githubusercontent.com/daanishrasheed/DATA607/refs/heads/main/Project%201/tournamentinfo.txt"
```

```
raw_lines <- readLines(data_url)
```

```
Warning in readLines(data_url): incomplete final line found on
'https://raw.githubusercontent.com/daanishrasheed/DATA607/refs/heads/main/
Project%201/tournamentinfo.txt'
```

```
# Drop blank lines, separator lines (all dashes), and the two header lines
clean_lines <- raw_lines[
  str_trim(raw_lines) != "" &
  !str_detect(str_trim(raw_lines), "^-+$") &
  !str_detect(str_trim(raw_lines), "^(Pair|Num)")
]

length(clean_lines)
```

```
[1] 128
```

```
parse1 <- function(x) {
  line1 <- clean_lines[2 * x - 1]
  line2 <- clean_lines[2 * x]
  f1 <- str_trim(str_split(line1, "\\|")[[1]])
  f2 <- str_trim(str_split(line2, "\\|")[[1]])

  numpair <- as.integer(f1[1])
  name <- f1[2]
  points <- as.numeric(f1[3])
  state <- f2[1]
  pre_rating <- as.integer(str_extract(str_extract(f2[2], "R:\\s*\\d+"), "\\d+"))
  rounded <- f1[4:10]

  o <- rounded |>
    str_extract("\\d+") |>
    as.integer()
  o <- o[!is.na(o)]

  list(pair = numpair, name = name, state = state,
        points = points, pre_rating = pre_rating, opponents = list(o))
}

n <- length(clean_lines) / 2

players <- lapply(seq_len(n), parse1)
```

```

players_df <- bind_rows(lapply(players, function(p) {
  tibble(pair = p$pair, name = p$name, state = p$state,
    points = p$points, pre_rating = p$pre_rating)
}))

head(players_df, 10)

```

```

# A tibble: 10 × 5
  pair name                state points pre_rating
  <int> <chr>                 <chr>   <dbl>     <int>
1     1 1 GARY HUA             ON        6       1794
2     2 2 DAKSHESH DARURI      MI        6       1553
3     3 3 ADITYA BAJAJ        MI        6       1384
4     4 4 PATRICK H SCHILLING MI       5.5       1716
5     5 5 HANSHI ZUO          MI       5.5       1655
6     6 6 HANSEN SONG          OH        5       1686
7     7 7 GARY DEE SWATHELL    MI        5       1649
8     8 8 EZEKIEL HOUGHTON    MI        5       1641
9     9 9 STEFANO LEE         ON        5       1411
10    10 10 ANVIT RAO          MI        5       1365

```

The above code chunk returns the pair #, name, state, points, and pre-rating for each person in the .txt file.

```

ratings <- setNames(players_df$pre_rating, players_df$pair)

avg_opp <- sapply(players, function(p) {
  opp_ratings <- ratings[as.character(p$opponents[[1]])]
  opp_ratings <- opp_ratings[!is.na(opp_ratings)]
  if (length(opp_ratings) == 0) return(NA_real_)
  round(mean(opp_ratings))
})

players_df$avg_opponent_pre_rating <- avg_opp

head(players_df, 10)

```

```

# A tibble: 10 × 6
  pair name                state points pre_rating avg_opponent_pre_rating
  <int> <chr>                 <chr>   <dbl>     <int>             <dbl>
1     1 1 GARY HUA             ON        6       1794             1794
2     2 2 DAKSHESH DARURI      MI        6       1553             1553
3     3 3 ADITYA BAJAJ        MI        6       1384             1384
4     4 4 PATRICK H SCHILLING MI       5.5       1716             1716
5     5 5 HANSHI ZUO          MI       5.5       1655             1655
6     6 6 HANSEN SONG          OH        5       1686             1686
7     7 7 GARY DEE SWATHELL    MI        5       1649             1649
8     8 8 EZEKIEL HOUGHTON    MI        5       1641             1641
9     9 9 STEFANO LEE         ON        5       1411             1411
10    10 10 ANVIT RAO          MI        5       1365             1365

```

1	1	GARY HUA	ON	6	1794	1605
2	2	DAKSHESH DARURI	MI	6	1553	1469
3	3	ADITYA BAJAJ	MI	6	1384	1564
4	4	PATRICK H SCHILLING	MI	5.5	1716	1574
5	5	HANSHI ZUO	MI	5.5	1655	1501
6	6	HANSEN SONG	OH	5	1686	1519
7	7	GARY DEE SWATHELL	MI	5	1649	1372
8	8	EZEKIEL HOUGHTON	MI	5	1641	1468
9	9	STEFANO LEE	ON	5	1411	1523
10	10	ANVIT RAO	MI	5	1365	1554

To get the average opponents rating for each player, the code builds a lookup table so any rating can be found by a player's number. Then for each player, it grabs their opponents' ratings, drops any missing ones and averages the data that is available.

```
write.csv(players_df, "tournament_ratings.csv", row.names = FALSE)
```

Conclusion

This parser will take this fixed-width data structure and convert it into a CSV with all of the relevant data formatted nicely in tabular format with the columns, name, state, total score, rating before the tournament and the average rating of their opponents before the tournament, all for each player. The hardest part of doing this was calculating the average of the opponents' rating as I had to find a way to access the pair number of each opponent. I wasn't able to do this with the data as a number, so I had to reformat it as a string and individually extracted each of them.